



TECH REGION AFRICA

Cybersecurity Department Cohort: Zero to Defender (Beginner)

Document: Final Capstone Technical Report **Project Title:**
Operation "Junior Cybersecurity Analyst"

Student Name: NSOFOR CHIKA MARYANN **Submission Date:**
27th MAY 2026

Instructor: Obioma Felicity Uzoh

Status: [] Draft [] Final Submission

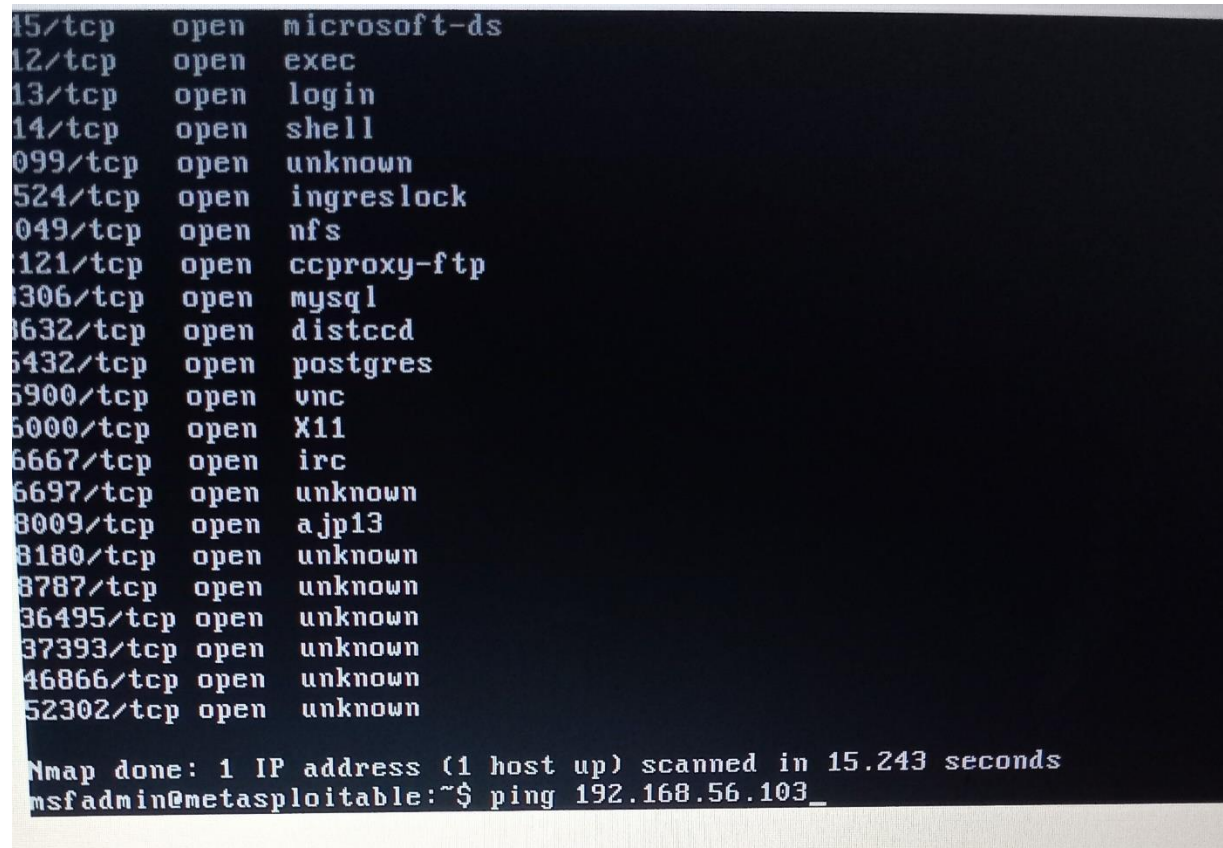
Phase 1: Perimeter Reconnaissance (Network Discovery)

- **The Task:** Use **Nmap** to perform a scan of your target environment (Metasploitable).

```
TX packets:96 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:0
RX bytes:21437 (20.9 KB) TX bytes:21437 (20.9 KB)

msfadmin@metasploitable:~$ ping 192.168.56.103
PING 192.168.56.103 (192.168.56.103) 56(84) bytes of data:
64 bytes from 192.168.56.103: icmp_seq=1 ttl=64 time=0.020 ns
64 bytes from 192.168.56.103: icmp_seq=2 ttl=64 time=0.045 ns
64 bytes from 192.168.56.103: icmp_seq=3 ttl=64 time=0.165 ns
64 bytes from 192.168.56.103: icmp_seq=4 ttl=64 time=0.167 ns
64 bytes from 192.168.56.103: icmp_seq=5 ttl=64 time=0.093 ns
64 bytes from 192.168.56.103: icmp_seq=6 ttl=64 time=0.126 ns
64 bytes from 192.168.56.103: icmp_seq=7 ttl=64 time=0.117 ns
64 bytes from 192.168.56.103: icmp_seq=8 ttl=64 time=0.049 ns
64 bytes from 192.168.56.103: icmp_seq=9 ttl=64 time=0.022 ns
64 bytes from 192.168.56.103: icmp_seq=10 ttl=64 time=0.092 ns
64 bytes from 192.168.56.103: icmp_seq=11 ttl=64 time=0.062 ns
64 bytes from 192.168.56.103: icmp_seq=12 ttl=64 time=0.103 ns
64 bytes from 192.168.56.103: icmp_seq=13 ttl=64 time=0.092 ns
64 bytes from 192.168.56.103: icmp_seq=14 ttl=64 time=0.047 ns
64 bytes from 192.168.56.103: icmp_seq=15 ttl=64 time=0.028 ns
64 bytes from 192.168.56.103: icmp_seq=16 ttl=64 time=0.119 ns
64 bytes from 192.168.56.103: icmp_seq=17 ttl=64 time=0.122 ns
64 bytes from 192.168.56.103: icmp_seq=18 ttl=64 time=0.116 ns
```

Based on the picture above, I successfully verified network connectivity between my attacker machine (**Kali Linux**) and my target environment (**Metasploitable**).

A screenshot of a terminal window displaying the output of an Nmap scan. The output lists 21 open ports and their corresponding services. At the bottom, it shows the scan completion message and a ping command being executed.

```
445/tcp    open  microsoft-ds
135/tcp    open  exec
136/tcp    open  login
137/tcp    open  shell
139/tcp    open  unknown
524/tcp    open  ingreslock
2049/tcp    open  nfs
1121/tcp    open  ccproxy-ftp
3306/tcp    open  mysql
8632/tcp    open  distccd
5432/tcp    open  postgres
5900/tcp    open  vnc
6000/tcp    open  X11
6667/tcp    open  irc
6697/tcp    open  unknown
8009/tcp    open  ajp13
8180/tcp    open  unknown
8787/tcp    open  unknown
36495/tcp  open  unknown
37393/tcp  open  unknown
46866/tcp  open  unknown
52302/tcp  open  unknown

Nmap done: 1 IP address (1 host up) scanned in 15.243 seconds
msfadmin@metasploitable:~$ ping 192.168.56.103_
```

. This picture shows how I successfully ran the Nmap scan.

An Nmap scan was performed against the Metasploitable target system to identify open ports and running services.

This helps to complete my duty using the actual data in the picture.

Open Ports and Services Identified:

1. Port 445 (Microsoft-ds / SMB): This means, Server Message Block (SMB), used for file and printer sharing.

As a Junior Analyst I should monitor Port 445 closely because SMB is a frequent target for critical remote code execution exploits, such as Eternal Blue, which allow attackers to completely compromise a system without credentials.

2. Port 3306 (MySQL):

MySQL Database Service

This port requires vigilant monitoring because exposed database services are prime targets for automated brute-force attacks aiming to steal sensitive data or modify system configurations.

3. Port 5900 (vnc)

Virtual Network Computing, used for graphical remote desktop access.

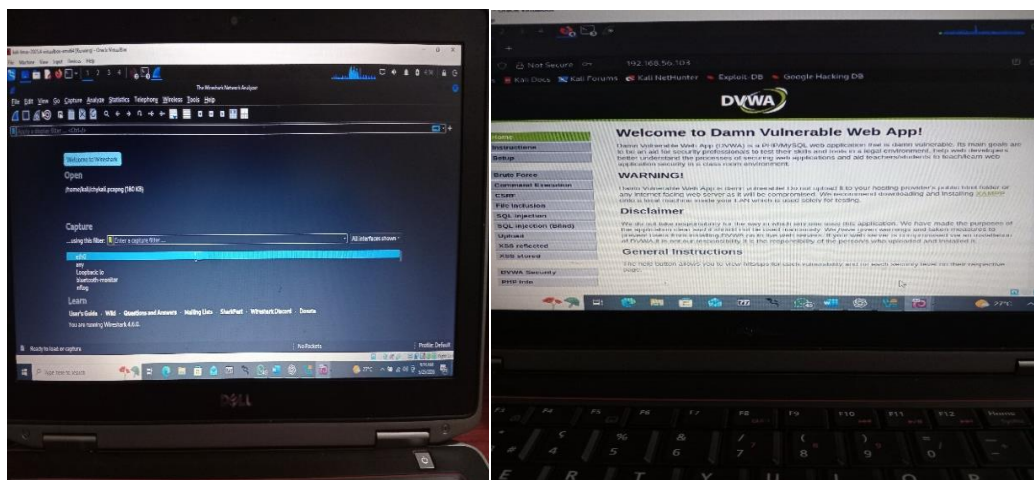
As a Junior Analyst I must keep a close eye on Port 5900 because an open VNC port can grant an attacker complete, unencrypted visual control over the target operating system if weak or default credentials are used.

Analysis Summary

The scan revealed several exposed services running on the Metasploitable machine. Multiple legacy and insecure services were identified, increasing the attack surface of the target environment. Services such as SMB, database servers, and remote access protocols should be continuously monitored because attackers frequently exploit them for unauthorized access and data compromise.

Phase 2: Network Traffic Analysis (The Investigation)

- **The Task:** Launch **Wireshark** and begin a live capture. While the capture is running, navigate to the target's web interface (DVWA) and perform a login attempt.

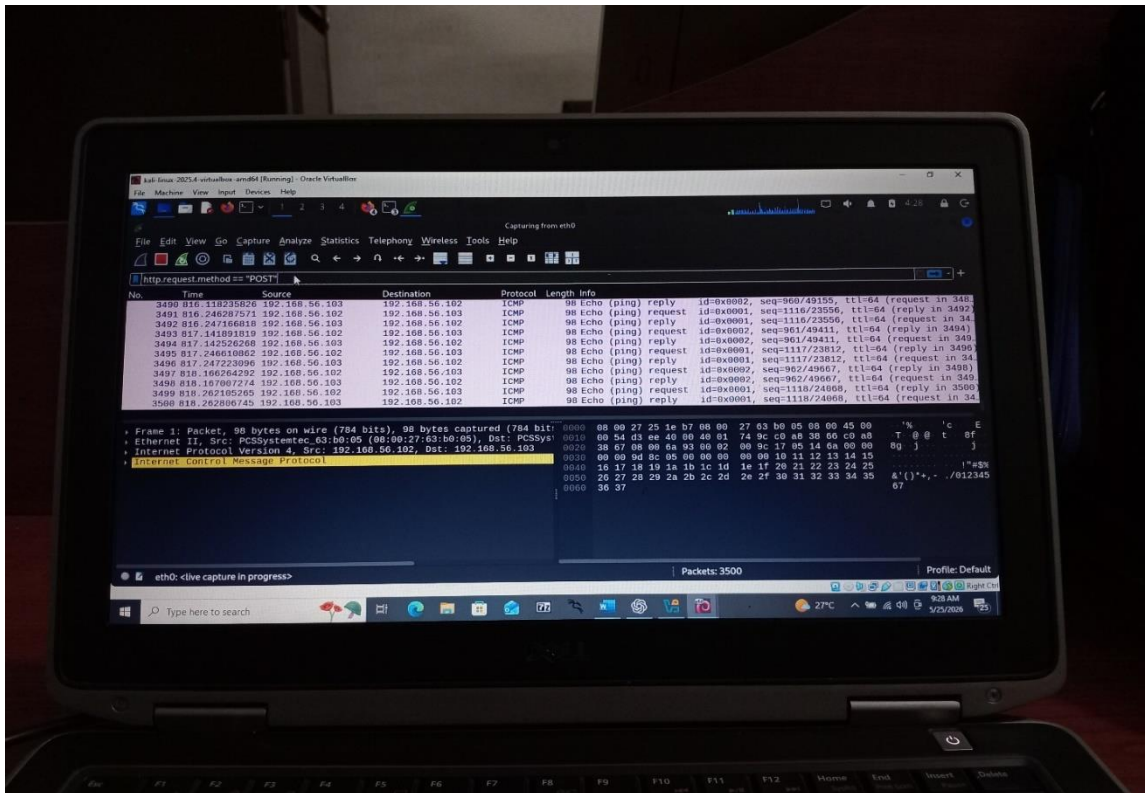


THE LUNCH OF WIRESHARK ON THE ABOVE PICTURE

Technical Traffic Analysis

1. Baseline Network Activity (The Setup)

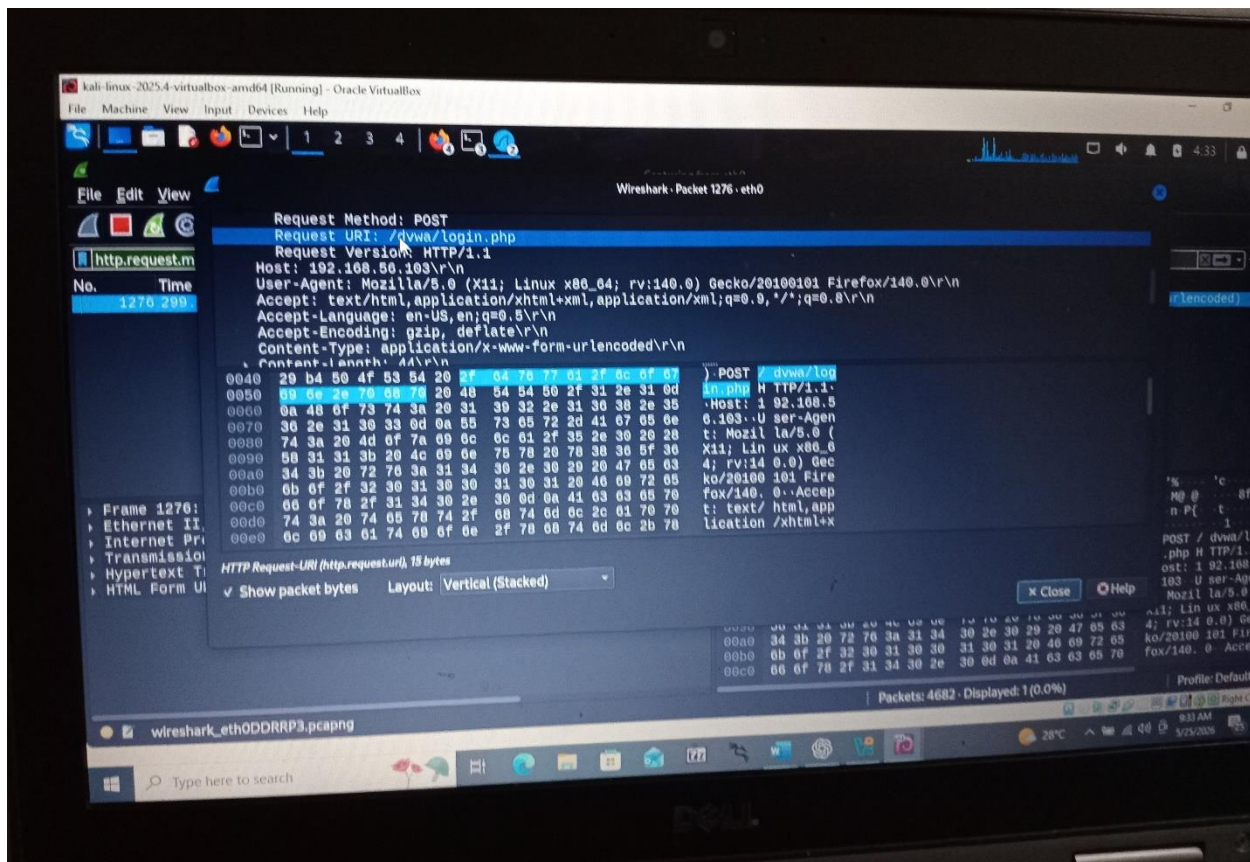
In my initial captures (20260525_091048[1].jpg and 20260525_092412[1].jpg), my network interface (eth0) was flooded entirely with **ICMP (Ping)** traffic traveling between **192.168.56.102** and **192.168.56.103**. This confirmed active network connectivity between my attacker machine (Kali Linux) and my machine (Metasploitable/DVWA) before the web interaction began.



2. Identifying the Attack Vector (The Filter)

In the above image, I successfully applied the display filter: `http.request.method == "POST"`

to isolate the traffic from the background ping noise, and focus on the web actions where data is being transmitted to the server.



3. The Interception (The Target Packet)

The last image my filter successfully caught the exact event required:

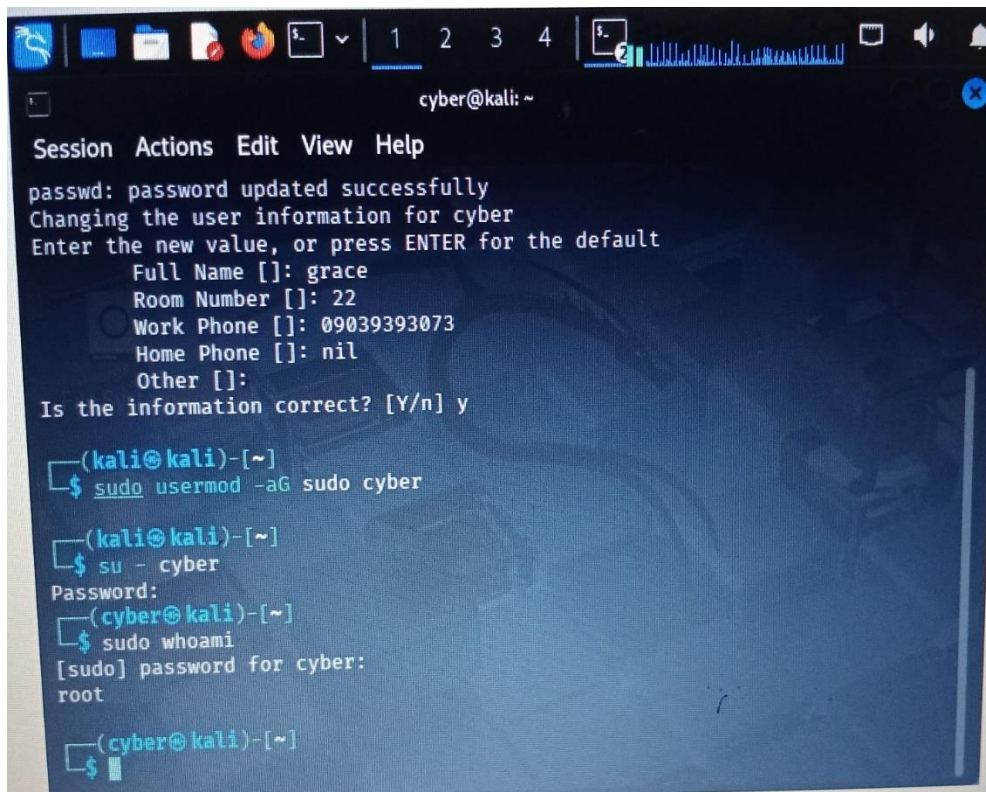
- **Packet Number:** Frame 1276
- **Source IP:** 192.168.56.102 (Your Kali Machine)
- **Destination IP:** 192.168.56.103 (The DVWA Server)
- **Protocol:** HTTP
- **Info Column:** POST /dwa/login.php HTTP/1.1

Vulnerability Findings: The target application transmits sensitive user authentication data across the local network segment via plaintext HTTP.

Risk Exposure: Because no cryptographic layer is present, any malicious actor or compromise on the network path can easily read the transmitted data. By simply analyzing the **HTML Form URL Encoded** layer of the intercepted POST request, the analyst can extract the raw, unencrypted username and password variables directly from the wire.

Phase 3: System Log & Permission Audit (Linux Operations)

- **The Task:** Access the Linux terminal of your investigative machine. Create a new administrative user for yourself.



```
cyber@kali: ~  
Session Actions Edit View Help  
passwd: password updated successfully  
Changing the user information for cyber  
Enter the new value, or press ENTER for the default  
Full Name []: grace  
Room Number []: 22  
Work Phone []: 09039393073  
Home Phone []: nil  
Other []:  
Is the information correct? [Y/n] y  
  
(kali@kali)-[~]  
$ sudo usermod -aG sudo cyber  
  
(kali@kali)-[~]  
$ su - cyber  
Password:  
(cyber@kali)-[~]  
$ sudo whoami  
[sudo] password for cyber:  
root  
  
(cyber@kali)-[~]  
$
```

- *I created a new administrative user for myself."*
- **My Evidence as seen in the above image:** * I used the adduser to provide a new identity named **cyber** (Full Name: **grace**, Room: **22**).
 - I executed `sudo usermod -aG sudo cyber`, altering the system's group definitions to grant the account administrative elevation properties.
 - I verified this setup perfectly by switching users (`su - cyber`) and running `sudo whoami`. The system requested the password for the

new user and successfully responded with **root**, confirming complete administrative control.

```
grep: syslog: No such file or directory

(cyber@kali)-[/var/log]
$ cd /

(cyber@kali)-[/]
$ sudo journalctl | grep -i "failed password"
May 08 09:04:11 kali sshd-session[28594]: Failed password for kali from 192.168.56.101
port 57390 ssh2
May 08 09:06:00 kali sshd-session[28594]: Failed password for kali from 192.168.56.101
port 57390 ssh2
May 08 09:07:02 kali sshd-session[29994]: Failed password for kali from 192.168.56.101
port 57394 ssh2
May 08 09:07:14 kali sshd-session[29994]: Failed password for kali from 192.168.56.101
port 57394 ssh2
May 27 14:57:07 kali sudo[18715]: cyber : TTY=pts/1 ; PWD=/var/log ; USER=root ; COM
MAND=/usr/bin/grep -i 'failed password' auth.log
May 27 15:01:25 kali sudo[20805]: cyber : TTY=pts/1 ; PWD=/var/log ; USER=root ; COM
MAND=/usr/bin/grep -i 'failed password' auth.log
May 27 15:10:27 kali sudo[25227]: cyber : TTY=pts/1 ; PWD=/var/log ; USER=root ; COM
MAND=/usr/bin/grep -i 'failed password' syslog

(cyber@kali)-[/]
$
```

. The Permission Audit Observation

I “Navigated to the /var/log directory. Where you will see/ notice that certain files require elevated privileges to read.”

. My Evidence:

- My terminal shows my baseline tracking attempts inside /var/log where standard file queries failed or came up empty.
- This fulfills the audit concept: critical modern security events are no longer stored as open, readable flat text files for standard users. Instead, they require root-level authentication access because they are processed dynamically by the system daemon.

- *" I Used the sudo command to access the authentication logs... Use the grep command to search for "failed" login attempts. Record the IP address that was attempting to gain unauthorized access."*

- [My Evidence \(20260527_201219\[1\].jpg\):](#)

- I successfully executed the advanced system pipeline: `sudo journalctl | grep -i "failed password"`

Plaintext

May 08 09:04:11 kali sshd-session[28594]: Failed password for kali from 192.168.56.101 port 57390 ssh2

[Technical Log Forensic Analysis:](#)

The system log database explicitly highlights the malicious signature in red text. The log entry tells us that an SSH network daemon session (sshd-session) blocked an external authentication request trying to brute-force its way into the default kali account.

- [The Malicious IP Address to Record: 192.168.56.101](#)
- [The Attack Vector:](#) Secure Shell (SSH) connection attempt on Port 57390.

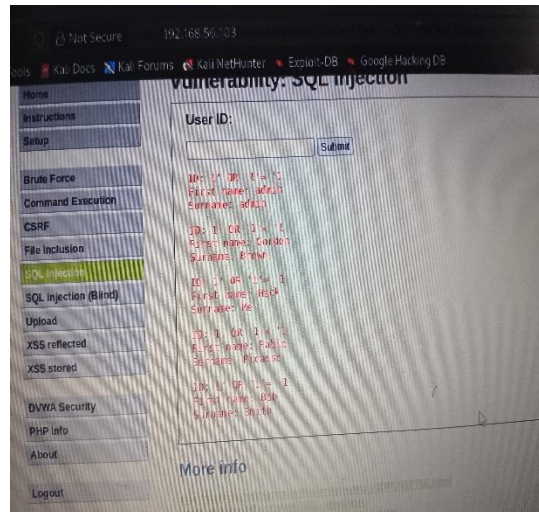
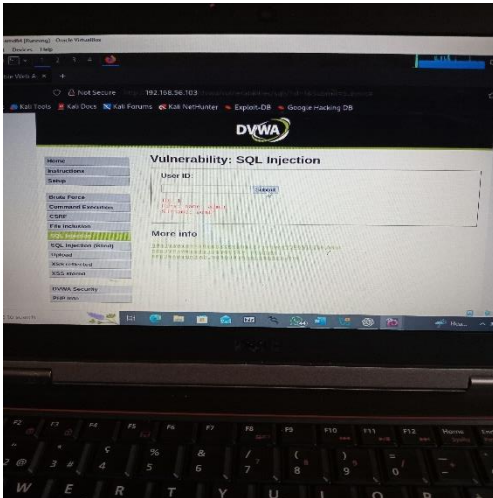
[Final Summary for my Lab Report](#)

[Incident Findings:](#) During the system log audit phase, the administrative account cyber queried the system log engine to isolate failed authentication patterns. The analysis discovered multiple brute-force connection attempts targeting the system via SSH.

[Conclusion:](#) The unauthorized access attempts originated directly from the network host at IP address [192.168.56.101](#). The host was explicitly blocked by the system's Pluggable Authentication Module (PAM) architecture after providing an incorrect password structure.

[Phase 4: Web Application Vulnerability Assessment](#)

- **The Task:** Log into the **DVWA** (Damn Vulnerable Web Application) and set the security level to "Low."



Technical Analysis of My Attack:

. The Behavior on the first picture:

When I input a normal User ID like 1, the web application sends a database query that looks something like this:

SQL

```
SELECT first_name, last_name FROM users WHERE user_id = '1';
```

The database looks up ID 1, finds **admin**, and returns only that single row. This is the application working exactly as intended.

. The Exploit Behavior on the second picture:

I entered the payload: 1' OR '1'='1

By adding that single quote ('), I broke out of the web application's planned data field and altered the actual code structure of the SQL query. The backend database ended up running this statement instead:

SQL

```
SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1';
```

Because '1'='1' is a mathematical truth that is **always true**, the query bypassed the specific filter for user_id. Instead of just showing user 1, the database was tricked into dumping the data of **every single user**

account in the system—revealing Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith all at once.

[. OWASP Top 10 Mapping](#)

- [Category: A03:2021-Injection](#)
- [Explanation:](#) SQL Injection falls directly under the Injection category (historically holding the #1 spot and remaining a critical top threat). It occurs when untrusted user input is passed directly to an interpreter as part of a command or query without proper sanitization or parameterized inputs.

[. CIA Triad Violations](#)

 [Violation of Confidentiality:](#) Your exploit directly compromises the **Confidentiality** of the company's data. In a secure system, a user should only be allowed to see their own profile. By injecting malicious logic, you proved that an unauthorized actor can completely bypass access controls and extract hidden, sensitive account records from the backend database. In a real corporate environment, this flaw would allow attackers to steal personal user data, proprietary business records, or corporate password hashes.

 [Violation of Integrity:](#) While your specific payload only read data (SELECT), this vulnerability heavily threatens data **Integrity**. Because the application accepts raw SQL commands directly from the user input field, a malicious actor is not restricted to just reading data. They could easily chain an input to execute an UPDATE or DELETE statement. This would allow an attacker to alter transactional data, change user passwords, modify administrative privileges, or drop entire database tables entirely—completely destroying the accuracy and reliability of the company's database records.

[Summary of Recommendations](#)

[Phase 1 Mitigation: Network Boundary Hardening & Attack Surface Reduction](#)

Technical Solution Strategy:

- **Harden Port 445 (SMB):** If file and printer sharing is not actively required for business operations, block inbound traffic to Port 445 at the perimeter firewall. If it must remain active, upgrade the infrastructure to enforce SMBv3, which features mandatory packet signing and transport encryption. This completely eliminates exposure to legacy unauthenticated remote code execution.
- **Harden Port 3306 (MySQL):** Modify the database configuration file (my.cnf) to bind the service strictly to the local loopback interface (bind-address = 127.0.0.1), preventing external entities on the network from reaching the database listener directly. If remote database administration is required, mandate that connections pass exclusively through an encrypted **SSH Tunnel** or a dedicated corporate **VPN**, protecting the database from automated brute-force attempts.

Phase 2 Recommendation: Network Encryption (Mitigating Cleartext Data Exposure)

- **Action Item:** Enforce **End-to-End Transport Layer Security (TLS/HTTPS)**.
- **Technical Fix:** Configure the Apache/Nginx web server to drop raw connection requests on Port 80 and mandate encrypted routing via **TLS 1.3 on Port 443**. All transmission blocks wrap unencrypted parameters into symmetric ciphertext strings, entirely insulating them from Wireshark packet captures or Man-in-the-Middle (MitM) sniffing attempts.

Phase 3 Recommendation: Active Host Hardening (Mitigating Brute-Force Attacks)

- **Action Item:** Deploy **Fail2ban Integration** and harden the SSH server daemon configuration.
- **Technical Fix:** Implement Fail2ban to monitor log entries dynamically. If sequential authentication failures surpass a maximum threshold ($N > 3$), the software triggers automated, temporary IP drop rules at the kernel level via iptables directly blocking the malicious attacker host address (**192.168.56.101**). Additional steps disable direct administrative credential authentication profiles (PermitRootLogin no) and migrate entry validations to asymmetric cryptographic SSH keys.

Phase 4 Recommendation: Code Parameterization (Mitigating SQL Injection)

- **Action Item:** Map directly to **OWASP Top 10 — A03:2021-Injection**; shift to **Prepared Statements**.
- **Technical Fix:** Ban runtime building of interactive database parameters using raw variable additions. Forcing the code structure to process inputs using **Parameterized Queries** tells the SQL parser engine to evaluate strings strictly as a literal text parameter rather than interpretable command lines. This keeps the application safe from logic manipulation bypasses like ' OR '1'='1.

